

week-2, lesson-2

Task-1

Given the number 1234 print out every digit separately.

Task-2

Print the number in integer, octal and hexadecimal representation

Task-3

In C, a hexadecimal integer constant is preceded by 0x or 0X. Consider the constant RGB color literal 0xFFEF0D:

```
int rgbColor = 0xFFEF0D;
```

Extract the individual Red (highest byte: FF), Green (middle byte: EF), and Blue (lowest byte: 0D) channel values using only basic arithmetic division (/) and modulo (%) operators. (Hint: Each channel fits in a single byte, which has a base-256 scale).

Print the extracted channels in the following precise formats:

Red: Display in standard decimal format using %i.

Green: Display in octal notation with a forced leading zero using %#o.

Blue: Display in uppercase hexadecimal notation with a leading 0X prefix using %#X.

Task-3 Solution: Arithmetic Unpacking

Since hexadecimal bases operate in powers of 16, every two hexadecimal characters represent one byte ($16^2 = 256$).

The lowest byte (Blue) can be isolated by taking the value modulo 256: `rgbColor % 256` or `0x100`.

The middle byte (Green) can be isolated by first shifting down (dividing by 256) and then taking the remainder modulo 256: `(rgbColor / 256) % 256`.

The highest byte (Red) can be isolated by dividing by 256^2 (65536): `rgbColor / 65536`.

Task-4

C representation of floating-point numbers can contain minor precision deviations due to how they are stored internally.

```
float floatingVar = 331.79f;
```

```
double doubleVar = 331.79;
```

In C, floating-point literals are treated as double-precision double by default unless given the f or F suffix. Create an arithmetic expression using casting to calculate the absolute representation discrepancy (error) introduced by the single-precision float conversion compared to the double-precision double value. Print this rounding error in scientific notation displaying exactly six decimal places using %e.

Task-5*

C's boolean data type `_Bool` guarantees that assigning any nonzero value (positive or negative) stores exactly 1, whereas assigning 0 stores exactly 0. You are given three variables:

```
int A = 100;
int B = 200;
int selector = -97; // This could be changed to any integer, including 0
Without using any if-else or conditional operators:
```

Coerce the selector variable into a `_Bool` variable named `boolVar`.

Construct a mathematical expression using only basic arithmetic operators (+, -, *) that selects and assigns the value of A to a variable `result` if `boolVar` is true (1), and assigns B if `boolVar` is false (0).

Print the internal boolean state using %i and the resulting multiplexed value using %i.

Output for selector = -97:

```
boolVar representation (%i): 1
Selected Result: 100
```

Output for selector = 0:

```
boolVar representation (%i): 0
Selected Result: 200
```